

CullingGroup을 넣은 7개 방식 비교 예측

Unity `CullingGroup` 을 7번째 후보로 넣어 기존 6개와 같은 시뮬레이션(두 배치 · 655m 경로 · 1,311 지점)으로 다시 계산했다. 플레이 측정이 아니라 측정 전 예측이다.

요약

- `CullingGroup`은 판정 "방식"이 아니라 판정을 계산해 주는 "엔진 기능"이다. 무엇으로 판정하느냐(거리 밴드 / 가시성)에 따라 두 가지로 쓸 수 있고, 결과가 정반대다.
- ⑦ 거리 밴드로 쓰면 결과가 ⑤ 거리와 같음다 (두 배치 · 1,311 지점 전부). 밴드 {20, 25}를 두고 band 1에서 이전 상태를 유지하면 히스테리시스가 된다. 차이는 결과가 아니라 계산 위치뿐 — C# 루프 대신 엔진 컬링 단계, 메인 스레드엔 바뀐 스폰너만 콜백.
- ⑦' 가시성으로 쓰면 탈락한다. 화면 가장자리 0.2m에서 나타나고 사라지며, 1m 왕복 떨림 23%, 감지 거리 15m 안 몬스터가 꺼져 있는 지점이 경로의 51%. 구를 키워도 떨림은 남는다(가시성에는 히스테리시스가 없다).
- `CullingGroup`의 제자리는 존재 층이 아니라 틱 층이다. ⑤로 컨 평균 33마리 중 화면에 닿는 스폰너 몫은 14마리, 실제 화면 안은 3마리. 나머지는 행동 트리 · 애니메이터를 줄여도 안 보인다. 지금 몬스터 프리팹 3종의 Animator는 `AlwaysAnimate` .
- 결론 순서는 그대로다. 존재 판정은 ⑤ = ⑥ = ⑦ 공동 1위(결과 동일). 셋 중 선택은 규모 · 제약으로 — 스폰너 23개에선 ⑤가 가장 단순하고, ⑦은 카메라 의존 · 1프레임 지연 · 게임 밖 측정 불가를 대가로 큰 규모에서 메인 스레드 비용을 줄인다.

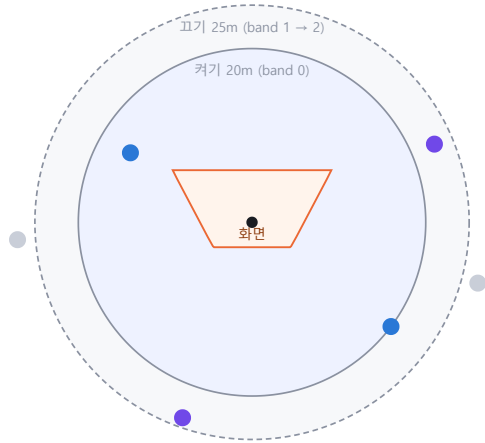
CullingGroup이 하는 일

Unity 매뉴얼 "CullingGroup API" · 스크립트 레퍼런스 "SetBoundingDistances" 기준 (2026-10-07 확인)

항목	API	이 비교에서의 의미
입력	<code>SetBoundingSpheres</code> — 배열을 복사하지 않고 참조	몬스터가 아니라 스폰너(고정 · 23개)를 등록 → 인덱스 = 스폰너, 안 바뀜
거리 밴드	<code>SetBoundingDistances</code> + <code>SetDistanceReferencePoint</code> . 거리를 정수 밴드로 양자화. 구 중심이 아니라 구 표면까지. 마지막 값 너머는 강제로 비가시	{20, 25} → band 0 켜 · 1 유지 · 2 끄 = ⑤의 켜기 · 끄기. 반경 0 구면 ⑤와 같은 거리
가시성	<code>targetCamera</code> 하나의 절두체 + 정적 오클루전만. 동적 오클루더는 반영 안 함	쿼터뷰라 사실상 "화면 사다리꼴에 닿느냐"
갱신 시점	카메라 컬링이 실행될 때만 계산. 바뀐 구만 <code>onStateChanged</code> , 풀링은 <code>QueryIndices</code> · <code>GetDistance</code>	결과가 렌더링 단계에서 나옴 → 스폰은 다음 <code>Update</code> 로 = 1프레임 지연
히스테리시스	문서에 없음	밴드 두 개 사이를 "유지"로 읽어 직접 만든다. 가시성에는 만들 수 없다(그룹 두 개 필요)
매뉴얼 사례	보이는 개체만 군중 시물 · 스폰 포인트가 안 보일 때 스폰 · 거리별 LOD 행동 · 1만 개 이상 근접 감지	"보이는 것만 시물" · "LOD 행동" = 틱 층, "근접 감지" = 존재 층

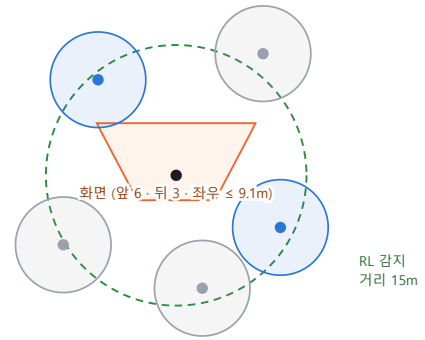
같은 API, 두 가지 사용법 (같은 축척)

㉦ 거리 밴드 — 존재 판정으로 사용



● 컴 ● 이전 상태 유지 (히스테리시스) ● 폼
SetBoundingDistances({20, 25}) · 반경 0 구 · 기준점 = 플레이어

㉦' 가시성 — 존재 판정으로 사용



구 반경 5.5m(스포너 퍼짐) · 화면 절두체에 닿으면 컴(파랑), 벗어나면 폼(회색)
회색 = 감지 거리 안인데 존재하지 않음

왼쪽은 플레이어 기준 원 두 개로 켜고 끈다(㉦와 같은 모양). 오른쪽은 화면 사다리꼴 기준으로 켜고 끈다 — 화면이 감지 거리 원보다 훨씬 작아서, 공격해 와야 할 몬스터 상당수가 존재하지 않는다.

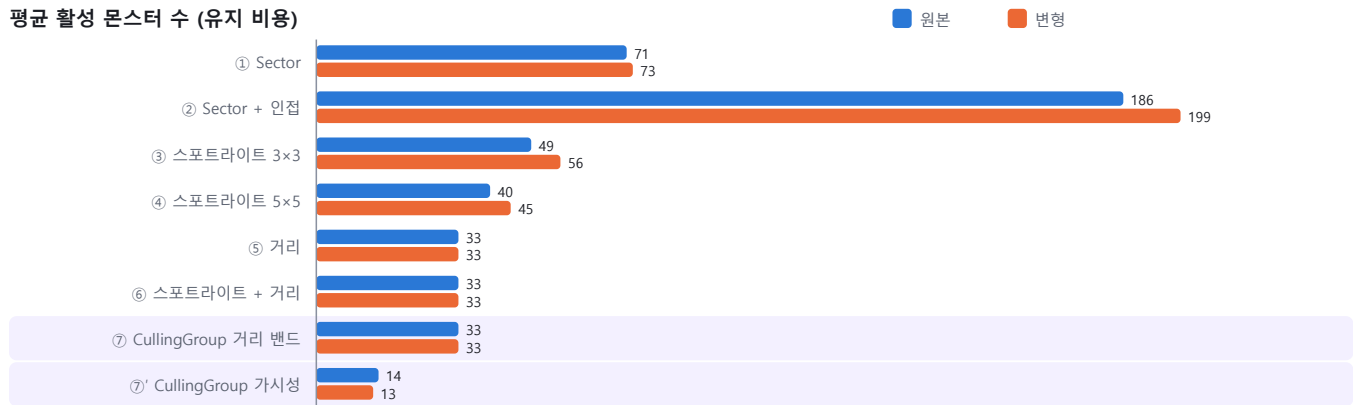
결과 — 원본 / 변형

각 칸은 "원본 / 변형". 초록 = 통과 · 주황 = 탈락 조건에 걸림 · 보라 = 경미. 기준 6개 숫자는 Predictions.pdf와 같다

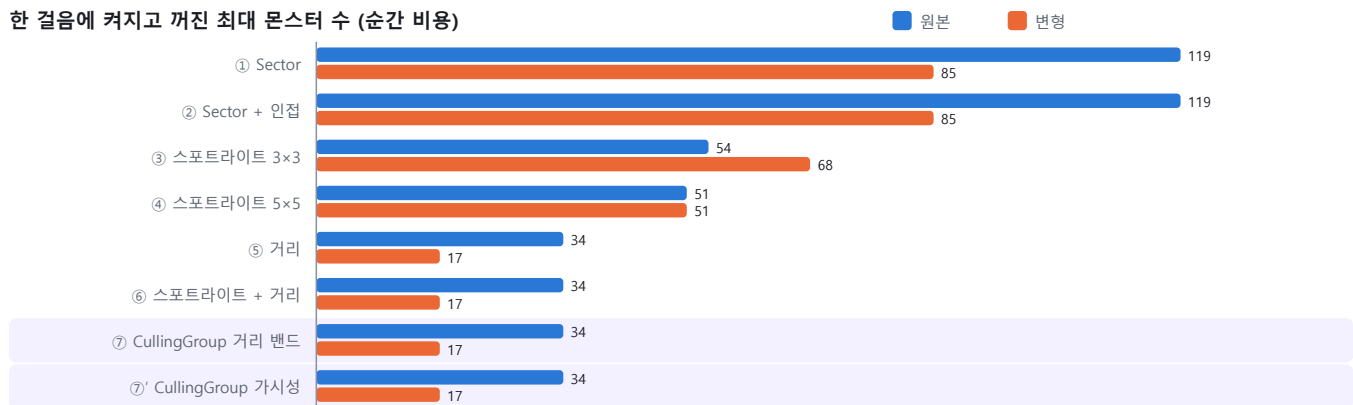
방식	평균 활성	최대 활성	한 걸음 최대 변경	1m 왕복 떨림	화면 가장자리 최소 여유 (m)	감지 15m 안 부재
㉠ Sector	71 / 73	187 / 153	119 / 85	2% / 2%	0.0 / 0.0	2.5% / 3.4%
㉡ Sector + 인접	186 / 199	295 / 295	119 / 85	2% / 2%	24.5 / 24.5	0.0% / 0.0%
㉢ 스포트라이트 3×3	49 / 56	102 / 85	54 / 68	11% / 11%	8.3 / 8.3	0.0% / 0.0%
㉣ 스포트라이트 5×5	40 / 45	102 / 68	51 / 51	14% / 14%	7.8 / 8.1	0.0% / 0.0%
㉤ 거리	33 / 33	102 / 68	34 / 17	0% / 0%	3.6 / 3.6	0.2% / 0.2%
㉥ 스포트라이트 + 거리	33 / 33	102 / 68	34 / 17	0% / 0%	3.6 / 3.6	0.2% / 0.2%
㉦ CullingGroup 거리 밴드	33 / 33	102 / 68	34 / 17	0% / 0%	3.6 / 3.6	0.2% / 0.2%
㉦' CullingGroup 가시성	14 / 13	51 / 51	34 / 17	23% / 22%	0.2 / 0.2	51.0% / 49.0%

새 열 두 개. 기준 "화면 안 등장·소멸"은 0.5m 간격 경로 위에서 세서, 정확히 화면 가장자리에서 생기는 경우를 놓친다(㉦'가 0회로 나옴). 그래서 생성 · 회수 순간 몬스터와 화면 사다리꼴 사이 최소 거리로 바꿨다 — 몸통이 1m 안팎이라 1m 미만이면 가장자리에서 튀어나오는 게 보인다고 판정(㉠의 0.0 = 화면 안 소멸, 기준 P7과 같은 결론). 감지 15m 안 부재 = RL 감지 거리 안인데 꺼진 몬스터가 1마리라도 있는 경로 지점 비율.

평균 활성 몬스터 수 (유지 비용)



한 걸음에 켜지고 꺼진 최대 몬스터 수 (순간 비용)



⑦'의 평균 활성이 가장 낮은 건 장점이 아니다 — 있어야 할 몬스터를 안 켜기 때문이다(감지 거리 안 부재 51%).

⑦' 가시성은 왜 탈락하나

구를 키우면 여유와 부재는 풀리지만, 떨림은 구 크기와 무관하게 남는다

⑦' 가시성 (원본 / 변형)	평균 활성	1m 왕복 떨림	가장자리 최소 여유(m)	감지 15m 안 부재
구 반경 5.5m	14 / 13	23 / 22%	0.2	51 / 49%
구 반경 10m	23 / 21	18 / 18%	4.2	17 / 17%
구 반경 15m	32 / 34	24 / 26%	9.4	0 / 0%

① 화면이 너무 작다

쿼터뷰 카메라

보이는 땅은 앞 6m · 뒤 3m · 좌우 ≤ 9.1m로 RL 감지 15m보다 좁다. 구 5.5m로는 감지 거리 안 몬스터가 자주 없다. 15m까지 키우면 풀리지만, 그때는 사실상 "화면을 중심으로 한 거리 판정"이다.

② 커는 선 = 끄는 선

떨림 23~24%

가시성은 참/거짓 하나라 "유지" 구간이 없다. ③④ 사각형 창(11~14%)보다도 떨린다. 막으려면 반경이 다른 그룹 두 개를 돌려야 하고, 그러면 ⑤를 두 번 하는 셈이다.

③ 존재가 카메라에 묶인다

게임플레이 결합

연출 카메라 전환 · 흔들림 · 줌이 몬스터 존재를 바꾼다. 메인 카메라가 렌더링하지 않는 동안(로딩 커버 등)은 갱신이 멈춘다. 존재 판정은 플레이어 위치만 봐야 재현된다.

매뉴얼의 "안 보일 때 스폰"은?

보조 규칙으로만

"보일 때 컴"이 아니라 "켜야 하는데 지금 보이면 미룸"으로 쓰는 것. ⑤는 등장 여유가 이미 3.6m 이상이라 필요 없다. 커기 반경이 화면보다 작아지는 장르(먼 시점 · 줌 아웃)에서 쓸모가 생긴다.

⑤ · ⑥ · ⑦ — 결과는 같고 "어디서 계산하나"만 다르다

⑤ = ⑥ 같음 · ⑤ = ⑦ 같음 (두 배치 · 1,311 지점, 지연 0으로 계산)

	⑤ 거리	⑥ 스포트라이트 + 거리	⑦ CullingGroup 거리 밴드
판정 계산	매 프레임 C# 루프, 스폰너 N개	창 안 후보만(약 4개) + 칸 바뀔 때 후보 갱신	엔진 컬링 단계(네이티브). 메인 스레드는 바뀐 스폰너 콜백만
N = 23	μs 단위	μs 단위	μs 단위 (+ 콜백 경로 고정 비용)
N = 10,000	N에 비례해 증가	창 안 개수에 비례 → 평평	메인 스레드는 변경 수에 비례 → 평평 (엔진 쪽 비용은 컬링에 포함)
반응 시점	같은 프레임	같은 프레임	1프레임 늦음 — 6m/s에서 약 0.1m, 등장 여유 3.6m 대비 무시 가능
의존	없음	체크 원점 · 칸 조건 (K-1)/2-S ≥ 끄기 반경	targetCamera가 렌더링해야 갱신 · 거리는 3D(이 맵은 높이 차 0.001m라 영향 없음)
수명 관리	없음	칸 등록(로드 1회)	구 배열 참조 유지 · 씬 전환 시 Dispose — 리소스 수명 체계에 편입
측정 · 검증	게임 밖 벤치 가능	게임 밖 벤치 가능	게임 밖 · 헤드리스 벤치 불가(카메라 렌더링 필요) · 동등성 로고는 1프레임 어긋남 감안
이음새	위치 → 집합, 그대로	그대로	위치는 기준점 갱신에만, 집합은 콜백이 유지 → SelectSpawners 는 복사만

이음새(ActivationPolicy)에 끼우면 — 스케치, 구현 아님

```
public class CullingGroupActivation : ActivationPolicy
{
    CullingGroup _group; BoundingSphere[] _spheres; int[] _ids; // 배열은 참조됨 → 필드로 붙잡아 둔다
    readonly HashSet<int> _on = new();
    readonly Queue<CullingGroupEvent> _pending = new();

    public override void Init(ActivationLayout layout)
    {
        // _spheres = 스포너 중심 · 반경 0 (㉟와 같은 거리) _ids[i] = 스포너 id
        _group = new CullingGroup { targetCamera = Camera.main };
        _group.SetBoundingSpheres(_spheres);
        _group.SetBoundingSphereCount(_spheres.Length);
        _group.SetBoundingDistances(new[] { 20f, 25f }); // band 0 컴 · 1 유지 · 2 끄
        _group.onStateChanged = e => _pending.Enqueue(e); // 컬링 중 호출 → 여기서 스포너하지 않는다
    }

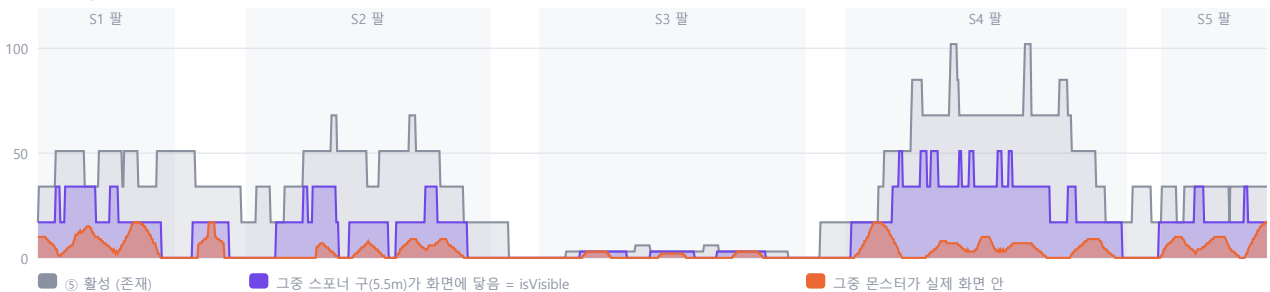
    public override void SelectSpawners(Vector3 pos, IReadOnlyCollection<int> current, HashSet<int> result)
    {
        _group.SetDistanceReferencePoint(pos); // 이번 위치는 다음 컬링에 반영 (1프레임)
        while (_pending.TryDequeue(out var e))
        {
            if (e.currentDistance == 0) _on.Add(_ids[e.index]);
            else if (e.currentDistance == 2) _on.Remove(_ids[e.index]); // 1이면 그대로 = 히스테리시스
        }
        result.UnionWith(_on);
    }
    // 씬 Clear 때 _group.Dispose()
}
```

첫 프레임 초기 상태가 콜백으로 오는지는 확인 필요 — 안 오면 Init 직후 `GetDistance` 로 한 번 동기화. 바뀐 게 없으면 콜백 0회.

CullingGroup의 제자리 — 틱 총

존재는 ㉟(또는 ㉟)가 정하고, 커진 몬스터를 얼마나 돌릴지는 가시성이 정한다

원본 경로 — ㉟로 컨 몬스터 중 “제대로 돌려야 하는” 몬스터



㉟로 컨 몬스터 (경로 평균)	원본	변형	의미
활성	33	33	존재 층의 결과. 교전 · 리쉬를 위해 화면 밖에도 있어야 한다
스포너 구가 화면에 닿음	14 (44%)	13 (40%)	스포너 단위 <code>isVisible</code> — 존재용 등록을 그대로 쓰는 거친 기준
몬스터가 실제 화면 안	3 (11%)	4 (11%)	몬스터 단위 구(매 프레임 위치 갱신)면 여기까지 내려간다

지금 바로 보이는 것

프리팸 확인

Droid_Helmet_AR · RL · Tank 프리팸 Animator
`m_CullingMode: 0 = AlwaysAnimate`. 화면 밖 몬스터도
 뼈대 계산을 전부 한다. `CullUpdateTransforms` 는 상태 기
 계 · 이벤트는 돌리고 뼈만 생각한다. RL은 애니메이션 이벤
 트로 발사하므로 `CullCompletely` 는 안 된다.

이 트랙에서의 위치

비교 이후

틱 층은 존재 판정 비교의 변수가 아니다(어느 방식에나 똑같
 이 었한다). 비교를 오염시키지 않도록 Phase 5 채택 이후로.
 Phase 1 "1마리 비용 분해"에서 애니메이션 · 행동 트리 묶이
 크게 나오면 다음 작업 근거가 된다.

예측 문장 추가 (P10~P14)

#	예측	근거 (원본 / 변형)
P10	⑦ 거리 밴드의 활성 집합은 1프레임 지연을 빼면 ⑤와 같다	같음 / 같음 (1,311 지점)
P11	⑦' 가시성은 탈락한다 — 가장자리 등장, 떨림, 감지 거리 안 부재	여유 0.2m · 떨림 23 / 22% · 부재 51 / 49%
P12	스포너 23개에선 ⑤ · ⑥ · ⑦의 선택 비용 차이가 결론에 영향 없다 (모두 μ s). 10,000개에선 ⑤만 비례 증가, ⑥ · ⑦은 평평	Phase 4 벤치 — ⑦은 빈 씬 + 카메라 렌더링 상태에서 따로 켜다
P13	존재를 ⑤로 정해도 커진 몬스터의 절반 이상은 화면 밖이다 → 틱 층 절감 여지	화면에 닿는 스폰너 몹 44 / 40% · 실제 화면 안 11 / 11%
P14	순위는 그대로 — ② > ① > ③ > ④ > ⑤ = ⑥ = ⑦ (유지 · 순간 비용)	평균 33 = 33 · 스파이크 34 = 34 / 17 = 17

부수 발견 — 커기 반경 20m가 0.5m 짧다 (⑤ · ⑥ · ⑦ 공통)

커기 20m는 "화면 9.1 + 스폰너 퍼짐 5.5 + 여유"로 정했지만, RL 감지 15m + 퍼짐 5.5 = 20.5m라 경로의 0.15% 지점에서 감지 거리 안 몬스터 1마리가 꺼져 있다(위 표 보라). 커기 21m / 끄기 26m면 0이 되고, 평균 활성은 33 → 34(원본) · 36(변형)으로 조금 늘 뿐이다. 0-1 결정 표를 고정하기 전에 반영할지 정할 것.

계획서 03에 반영할 것 (제안)

- 후보 표에 ⑦ **CullingGroup 거리 밴드** 추가 — 메인 측정(Phase 3)에 놓되, 결과는 ⑤와 같을 것이므로 볼 것은 선택 비용 · 반응 지연.
- ⑦' 가시성은 구현하지 않고 이 시뮬레이션으로 탈락 처리(근거 기록).
- Phase 4 선택 비용 벤치: 곡선 3개 = ⑤ C# 루프 · ⑥ 격자 · ⑦ CullingGroup(02_Plan 7-4의 원래 구상). ⑦은 게임 밖 C# 벤치로 못 재므로 빈 씬 + 카메라로 따로.
- Phase 2-5 동등성 검증에 ⑤ = ⑦ 추가(1프레임 어긋남 허용).
- 틱 층(가시성 · Animator 컬링)은 이 비교 밖, Phase 5 이후 후속.